

面向 AI Agent 轨迹的可配置语义剖析

AgentSight 研究原型

2026 年 7 月 4 日

摘要

AI agent 的执行历史已经不再只是一次 prompt 或一棵 trace tree。一个真实轨迹可能同时包含用户意图、LLM 响应、tool call、API 调用、GUI 动作、浏览器动作、进程、文件、网络、计划步骤、subagent 以及 benchmark 给出的成功、失败、安全、重复和冗余标签。现有 agent observability 工具通常把这些对象固定成 prompt、session 或 span 层级，传统 profiler 则固定成函数或进程层级。这两种做法都会把 profiling 问题过早绑定到某一种边界上，使同一段轨迹很难按任务、阶段、工具、动作、质量标签或人工分组递归折叠。

本文提出一种更小的语义剖析模型，只保留 operation 和 operation stack 两个核心抽象。operation 是带字段和权重的观测。Prompt、tool call、process、syscall、GUI action 和 plan step 可以是 operation 记录或派生字段，session、span、task 和 trace id 则是 operation fields、交换容器或 baseline shape，不是新的 profiler 对象。operation stack 是用户从字段中选择的递归投影，mapping 和 tagging 在 stack 构造前派生字段，而不是创建第三种 profiler 对象。当用户不想给出固定 stack 字段链时，agentpprof 也可以从可见 evidence、semantic shift 和 query hint 中给相邻 operation boundary 打分，递归诱导多层 operation frames，再用 operation-only stack 折叠。关键点是聚合不是第三个对象，也不是固定 trace tree。Query time 选择的 stack 形态决定聚合结果。因此本文的新意不在单纯 query-time aggregation 或 flamegraph renderer。它在于把 agent operation record、递归多字段 operation-stack 投影和 hidden-label profile-group scoring 放进同一个 profiler 评估问题。我们在 Rust agentpprof 中实现该模型，并用三个核心 profiling 实验和一个可重放性检查评估它。RQ1 验证泛化覆盖和递归 operation-stack 形成。在 15 个轻量采样的公开标注轨迹来源上，agentpprof 把 47,590 个 samples 投影到同一 operation layer。同一批 13,265 个 operations 可以从 9 个 dataset stacks 折叠为 57 个 phase stacks、226 个 semantic stacks、455 个 action stacks，或 3,757 个 fixed-session stacks，而不改变输入 operation。这说明 prompt、tool、GUI action、process 和人工边界都进入 operation layer，session/span 只作为字段、容器或对视图参与 stack 查询，不是新的 profiler 抽象。

RQ2 是 hidden-label localization/ranking 主实验。在 AgentRewardBench、SATraj-OS、AgentNet 和 OSWorld-Human 的 4 个 oracle-rich 数据集、6 个任务、34,539 个 operations 和 3,699 个 positives 上，我们把 profile groups 当作 ranked localization outputs，比较 flat、fixed-session drilldown、dataset-native、raw-action 和 operation-stack views。Task-query operation-stack profiling 的 top-5 median work 为 0.0937，而 flat summary 为 1.0。相对 fixed-session drilldown，它在 5/6 tasks 上提高 top-5 recall，并

把 median groups 从 285.0 降到 157.5。

RQ3 隔离机制和 profile-configuration actionability。Rank feature、stack depth、mapping、transfer、diagnostic lens、case view 和 counterfactual 分析显示，有效配置随 task 和 objective 变化，operation-stack 是 Pareto point，不是 universal default。Executable profile-spec patches 在 5/6 tasks 上改善 AP、top-5 lift 和 first-positive work。Supervised held-out boundary-field ablation 把 OSWorld-Human AP 从 0.2402 提到 0.2583，并把 groups 从 108 降到 74，但保留 work 反例。Field-suitability synthesis 进一步把 mapping、stack-depth、rank-feature 和 boundary-field 证据变成 accept/caution/reject 配置决策。RQ4 验证可重放性、离线成本和复现边界。76 个 profile 配置在 deterministic replay 下保持语义和 byte-level 输出稳定，local-session、agent-trace 和 standard-trace 输入都通过同一 operation-stack path 重放。RQ4 只说明离线证据可重放，并约束数字、数据来源选择和主张边界，不作为第四个 empirical accuracy result。

因此本文支持的是有范围限定的 profiling 主张：operation-stack profiling 能在公开真实标注 trace 的 dataset-provided hidden labels 下定位、排序和解释任务相关 failure、quality problem 和 dataset-provided semantic boundary，并相对 flat summary 减少 inspection work、相对 fixed-session drilldown proxy 改善 median fragmentation tradeoff。本文不声称提升 human productivity，不声称自动发现所有 intent boundary，不声称 metric dominance，也不声称完整兼容现有 trace ecosystem。

1 问题

AI agent 轨迹的调试问题正在从单次调用回放变成跨任务剖析问题。开发者不只想知道某个 span 什么时候开始和结束，也想知道哪类任务导致最多重复点击，哪些 GUI 轨迹反复输入，哪个工具调用族对应失败，哪些安全攻击类型集中在某些操作阶段，或者一个 prompt 序列能否被折叠成 task、phase、action 和人工 subtask 等多个深度。如果 profiler 把 session、prompt 或 tool call 写死成核心层级，这些问题会被迫使用同一个边界。如果 profiler 只保留进程或函数栈，上层语义和 benchmark oracle 又会丢失。

本文的出发点是把 agent trace 中所有可剖析对象都降到同一层。本文所说的语义剖析，是指用 agent operation fields 在跨 session 轨迹上形成可聚合、可排序、可评分的诊断单元。一个 prompt 可以是 operation，一个 tool call 可以是 operation，一串 prompt 也可以通过 mapping 被派生为同一个 intent 或 task operation field。同样，GUI action、API call、browser action、process event、syscall、安全标签、step correctness 和人工 grouped-action id 都不是新的 profiler 抽象，而是 operation fields。Profiler 的核心任务不是决定唯一正确边界，而是让用户和实验可以在同一批 operations 上选择 stack 字段、派生 mappings，并得到不同深度的递归折叠。

这个问题有两个系统挑战。第一，抽象必须足够小，否则 prompt、session、tool call、plan、subagent 和 process 会各自形成独立 pipeline。第二，抽象又必须足够可配置，否则 flamegraph 只会成为固定视图，不能回答边界、失败、安全和质量诊断问题。本文的目标是证明 operation 和 operation stack 足以覆盖多类真实标注 agent 轨迹，并且 stack shape 与 aggregation 的选择会系统性改变质量、边界、定位、work 和碎片化分析结果。这个目标把贡献放在 profiler abstraction 和评估问题上，而不是放在某个单一 flamegraph 视图上。

2 设计

2.1 两个核心抽象

operation 是唯一的样本级对象。每个 operation 是一个带权重的字段集合，例如 op=prompt、op=llm、op=tool、tool=browser、action=click、phase=navigate、status=failure 或 step_correct=incorrect。这些字段可以来自本地 Codex/Claude session，也可以来自外部 benchmark 的标注轨迹。实现上，外部轨迹使用每行一个 JSON object 的 operation JSONL，并把原始任务文本、截图和长对话默认排除在提交复现包之外。

operation stack 是从 operation fields 中选择出的有序 frame 列表。例如同一批 operations 可以用 project,dataset 查看数据来源，也可以用 project,dataset,task,phase,op,tool,action,status 查看动作深度，或者加入 human_group、step_correct、safety 和 looping 做诊断视图。Stack 不是 trace tree，也不是 session hierarchy。自动 operation-stack induction 仍然只产生 operation stack: agentpprof 在当前连续 operation segment 内用可见 evidence、semantic shift、changed-field density 和 query hint 给相邻 cut 打分，要求左右 child 有足够权重和不同 dominant evidence，然后递归写入多层 operation frames。因此用户给的是 predicate、view 和 query hint，不是 analysis_task,phase,action,status 这样的固定 stack 字段链。它是一个查询结果，用户可以按问题改变深度和字段。

2.2 Mapping 和 tagging

Mapping 和 tagging 是派生 operation fields 的一等机制。它们在 stack 构造前运行，读取 operation 的 key=value 文本，并写入新的字段。例如 desktop action 中的 type 和 key 应先进入 phase=input，再考虑通用 web action verb。API/tool traces 应先进入 tool/API phase family，再处理 search 或 create 这类词。这一顺序来自跨 web、desktop、API/tool 和 step-quality 轨迹时暴露出的关键设计约束。如果通用 action rule 抢在 operation-family rule 前面运行，desktop、web 和 API 轨迹会被错误合并。

Mapping 不引入第三个抽象。它只是在 operation 上写字段，使同一个 stack specification 可以跨数据集复用。当前 Rust CLI 支持 inline --op-map FIELD:LABEL=REGEX 和文件形式 --op-map-file。--where FIELD=REGEX 和 FIELD!=REGEX 在 mapping 之后、stack 构造之前选择 operation 子集，多条 predicate 取 AND。这个选择步骤实现形式化模型里的 ϕ ，不是新的 profiler 对象。同一规则格式可以从已标注 operations 中派生出来，再用于 held-out 和 leave-dataset-out 泛化测试。--profile-spec 把 operation files、mapping files、predicate、view、stack 和输出

路径打包成 JSON 配置，便于复现实验。命令行参数仍可覆盖 spec 默认值。--deterministic-output 或 spec 中的 deterministic_output 可以把 JSON generated_at 和 pprof profile time 固定为可复现值，用于 byte-stable replay。它不改变模型本身，只是把已有 operation 和 operation-stack 查询显式记录下来。

2.3 Views 和非火焰图分析

--view 决定采样和权重，例如 operation count、token、file、network 或 time。--stack 决定递归折叠形状。Folded stack 和 SVG flamegraph 是其中一种输出，但不是唯一输出。我们同时生成 JSON profile、stack tree、top-field table、parent-child transition、depth sweep、boundary F1、V-measure、coverage report、grouped-action report 和 HTML quality report。这些视图共享同一 operation 输入和 stack 语法，因此 failure、safety、looping、redundancy 和 human group 可以被当作普通字段评分，而不是另建 failure profiler 或 safety profiler。

3 实现

agentpprof 是当前被测实现。它是 Rust CLI，读取本地 Codex/Claude 历史或外部 operation JSONL，并输出 pprof-compatible profile、folded stacks、SVG 和 JSON。核心命令形态如下：

```
agentpprof --operation-file ops.jsonl --view operations
--stack project,dataset,task,phase,op,tool,
action,status --op-map-file operation-map.txt
--where task=verify
-o out.folded
```

外部数据集进入系统前先被规范化为 operation JSONL。规范化过程只下载或流式读取所需公开文件的采样前缀。对于包含截图、长 prompt 或完整对话的数据集，默认保存 redacted rows 和 operation JSONL，不提交原始数据。本地 agent session 通过 agent-session 解析为 agentsight.agent-session.trace.v1 trace。agentpprof 可以用 --export-trace 写出 filesystem-normalized trace，用 --trace-file 读回，也可以把 trace 转成 operation JSONL 后折叠。这个 replay path 验证 filesystem/tool-command portability 和 folded-output equality，但不声称 prompt 或 LLM preview 已完整脱敏。标准 trace 也只是 exchange container。agentpprof 可以用 --export-standard-trace 写出 Chrome/Perfetto-style traceEvents，再用 --standard-trace-file 导入为普通 operations 后折叠。在一个真实标注 operation 前缀上，512 operations 可以导出为 512 个 Chrome complete events，并在导入后仍得到 byte-identical 的 512-sample / 11-stack folded 输出。这说明 exchange path 不改变 operation samples 或 stack shape，但不提供新的准确性主张。质量评估复用 agentpprof 的 operation mapping contract，并计算字段覆盖率、oracle V-measure 和序列相邻边界 F1。同一 profile output 还可以生成 flamegraph 之外的 tree、top-kind 和 transition 报告。

这个实现刻意保留简单接口。它不像 jq 那样完整表达 JSON 查询语言，但它把 profiler 的关键自由度暴露为 flags 和可提交配置。数据源由 --trace-file 或 --operation-file 选择，权重由 --view 选择，查询子集由 --where 选择，字段派生由 --op-map 选择，递归边界由 --stack 或 --induce-operation-stack 选择，JSON group 排序由 --rank-rule、--rank-op-rule 和

--rank-mode 选择, 复现配置由 --profile-spec 固定。这些接口结果只说明 E1-E4 的实验配置可以经同一 profiler path 执行, 而不是形成额外主实验。三类实现证据共同界定这个接口。Mapping-derived where_rules 可以在 folding 前选择 729/729、714/714 和 4,285/4,285 个预期 operations。Profile specs 可以组合 operation source、predicate、visible rank rule、rank mode 和显式 stack depth, 并在 12/12 variants 中保持不引入 prompt/session frame。Standard-trace round trip 在真实标注 operation 前缀上保持 512 samples 和 11 stacks。这些结果支撑 E1 的接口边界, 不另起一个 experiment block。

Ranker 和 actionability 的实现证据被归入 E3。Stack-text rule 与 rank-mode variants 可以从 profile spec 复现, 但对 safety/side-effect 过粗。Operation-level visible feature density 在 folding 前匹配 mapped operation token, 并在 semantic stack 上使 AP 赢 5/6 tasks、top-5 lift 赢 4/6、first-positive work 赢 5/6, coarse stack 也在 AP 上赢 4/6。Leave-one-feature、robustness 和 transfer results 说明 repeat_signal、write-action、status=success 等字段在不同 family 中作用不同, 误导性 safety-like 或 input-phase features 会伤害 AP 或 first-positive work。Global/equal feature banks 常能保留收益, 但 guided repair 使用离线评分反馈, 因此不是 label-free automatic ranker。这些结果支撑 E3 的 mechanism/actionability 解释。

Profile-spec replay、输入来源 replay 和 deterministic output 归入 E4。76 个 profile specs 覆盖 view specs、rank-feature specs 和 robustness specs。Rust agentpprof --profile-spec 重复运行两次后 semantic profile hash 全部稳定, 默认 raw-byte hash 只有 4/76 稳定, 因为 JSON profile 包含生成时间戳。启用 --deterministic-output 后 raw-byte hash 达到 76/76。同一个 E4 复现检查进一步验证 profile spec 可以 replay session_files、trace_files、standard_trace_files、regex tag_rules、where_rules、rank rules、stack override 和 include_standard_trace_args。这些输入源最后仍归一成 operations, 并由 operation stack 折叠。这关闭的是输出 byte-stability 缺口, 不是新的 accuracy benchmark、live overhead benchmark 或第五个主实验。因此 prompt、session、toolcall、process、syscall、plan 和 subagent 都可以在同一套 mechanism 中出现, 而不是成为互不兼容的 object model。

4 实验方法

评估把 profiler 输出当作 ranked localization targets, 而不是把 flamegraph 图形本身当作结果。我们只使用公开的、别人已经标注好的 agent 轨迹或交互轨迹, 并通过 Hugging Face Dataset Viewer、HF repo JSONL、公开 GitHub JSON 和公开 benchmark artifacts 做轻量采样。这种数据策略避免同步完整测试集、图片归档和 gated 数据, 同时保留每个主张需要的顺序、动作、质量、安全或人工边界 oracle。

每个主张都对应一个可执行 oracle 和一个带范围的指标。Phase/action 结构用 V-measure 和相邻边界 F1 评分, grouped-action 用人工 group id 评分。Looping、safety、step correctness 和 redundancy 使用数据集原生标签, 递归深度使用同一批 operations 下 unique stack 和 compression 变化。我们从 AgentRewardBench、SATraj-OS、AgentNet 和 OSWorld-Human 的 operation JSONL 构造 6 个 oracle-backed analysis tasks, 并把 flat、fixed-session、

dataset-native hierarchy、raw action stack、operation-stack、label-drilldown oracle view 与 width、visible-risk、query-aware 和 oracle-upper-bound rankers 放在同一评分协议下比较。指标包括 precision@k、recall@operation-budget、F1@k、AUPRC-style average precision、nDCG、work-to-first-positive、groups-to-50% recall、group count 和 task-level profile-configuration insight。

Hidden-label scoring 把答案字段从 profiler input 中隔离出来。非 oracle policies 只读取 status、repeat_signal、phase、action 和 environment 等可见 operation fields, 不读取 looping、safety、step_correct 或 target_positive。因此 first-positive、first-enriched 和 high-lift results 都是同一 operation/operation-stack 输出的事后 oracle scoring, 而不是 automatic detection、人类效用或答案泄漏。

机制实验把 field derivation 和 boundary derivation 都限制在两个抽象内。监督式 adjacent-boundary backend 只写回 learned_group_pattern、learned_segment_pattern、learned_segment_position 和 learned_boundary_prev 等 operation fields, 递归折叠仍由 agentpprof 使用普通 --profile-spec 和 --stack 完成。每个候选标签家族还必须经过 suitability 和 simple-baseline comparison, 防止把 safety、history context 或 repetition proxy 错当作语义边界。负结果也保留在论文中。AgentNet 的 task status 很弱地预测 per-step correctness, repeat signal 也很弱地预测 step redundancy。这些反例说明 profiler 能承载诊断字段, 但简单 mapping 不能替代专门质量模型。

复现约束把论文主张连接回可重放结果。我们只把可重放 profile results 中已经存在的数字和数据来源选择写进论文主张与局限, 并用 profile specs 固定 operation source、mapping、ranker、stack 和 deterministic output 选项。这个设置让论文检验 profiler 自身是否能在真实 workload 上相对 dataset-provided hidden labels 定位、排序并指导优化, 而不是提前声称 analyst 使用它更快。

4.1 Hierarchy ablation baselines

本文比较的是 hierarchy choices, 而不是把每个已有 observability 产品完整复刻一遍。Flat profile 把每个任务压成一个 summary group, 用来检验 operation stack 是否比无层级摘要更 selective。Fixed-session drilldown 把 session 或 demo id 放进 stack, 用来模拟 session/prompt-first 调试界面和传统 trace-tree drilldown 的碎片化风险。Prompt/session tree、tool-call hierarchy 和 OTel/span-like trace tree 在本实验中都建模为不同 stack specs。它们仍然读取同一批 operations, 只是把 session、prompt、tool、process 或 trace id 放在更靠前的 frames。Label-drilldown 是 oracle-aware upper-bound style view, 它把 hidden label 直接放入 stack, 用来界定“如果答案字段已经可见”时的上限, 而不作为真实用户界面 baseline。这些 ablations 隔离边界选择对聚合、selectivity、recall 和 inspected work 的影响。因此本文不会声称 agentpprof 支配所有生产 observability UI, 只声称 operation stack 把这些边界选择统一成可配置查询。

5 结果

结果部分检验一个 profiler 主张。agentpprof 应该把异构 agent 轨迹统一为两个抽象, 用 operation-stack queries 定位 hidden-label positives, 并给出可执行的 profile-

表 1: 公开标注轨迹数据源和当前用途。15 个来源支撑 E1 覆盖; ScaleCUA 是补充点。RQ2 hidden-label 主结论依赖 AgentRewardBench、SATraj-OS、OSWorld-Human 和 AgentNet 四个 oracle-rich 数据源。

数据源	原生标注	转换方式	主要用途
WebLINX, Mind2Web, AgentTrek, WebShop	web action, demo/task, reward 或 action history	HF Dataset Viewer 或 HF repo shard	web navigation 和 shopping trajectories 的跨源覆盖。
API-Bank, ToolBench, tau-bench	API/tool call, solution path, 多轮 user/assistant/tool messages, outcome	HF rows 或 repo JSONL	tool/API/dialogue phases 和 user-agent-tool operation 形态。
SWE-agent	issue trajectory, command action, success target	HF Dataset Viewer	software-agent command 轨迹。
AndroidControl, GUI-Odyssey	mobile/cross-app step action 与 instruction	public/HF samples	移动 GUI action depth 和跨 app scale。
AgentRewardBench	expert success, side-effect, looping, optimality	annotation rows 加 cleaned BrowserGym steps	failure, looping, side-effect 的非 flame-graph 诊断。
SATraj-OS	desktop action, success, safety, reward, attack type	HF safety config	桌面安全和 attack-type operation fields。
OSWorld-Human	human single-action 和 grouped-action 轨迹	GitHub JSON files	人工 grouped-action 边界 oracle。
AgentNet	human desktop PyAutoGUI, task outcome, step correctness, redundancy	HF repo JSONL streaming	最大 human desktop step-quality oracle。
ScaleCUA	GUI navigation action, previous-operation context, gold action	HF annotation JSONL streaming	补充多步 GUI history-depth 覆盖。

configuration 建议。RQ1 先验证 operation layer、递归 folding、field derivation 和人工边界是否落在同一个模型内。RQ2 是主 hidden-label localization/ranking 实验, 直接比较 flat、fixed-session、dataset-native、raw-action 和 operation-stack views。RQ3 隔离 stack depth、ranker、mapping、transfer、case 和 profile-spec patch 的作用。RQ4 只检查 profile-spec 可复现性、离线成本和主张边界。其他分析用于解释反例和适用条件, 不扩大核心实验数量。

读者可以从三组证据判断该主张是否成立。表 1 和表 2 先给出 workload sources、oracles、主证据和范围。表 3 检查两抽象和递归折叠, 表 5 与图 1 左图给出 hidden-label fidelity 和 baseline tradeoff, 表 6 与图 1 右图给出 mechanism/actionability。表 7 和表 8 只给出 replay/cost 证据。Portfolio、case 和 verdict 视图解释这些主图表的数据来源、反例和适用边界。

5.1 RQ1/E1: 泛化覆盖、递归折叠和边界字段

RQ1 检查同一个 operation layer 能否覆盖异构 agent traces, 并避免把 prompt、session 或 tool 边界写死成 profiler 层级。它使用 public dataset labels、native trajectory fields、OSWorld-Human human boundary 和 AgentNet quality labels 作为 oracle, 比较 dataset-native stacks、no-map folding、fixed-session stacks、profile-spec replay 和 trace round trip。主要指标包括 operation coverage、unique-stack range、mapping compression、boundary F1/V-measure 和 replay equality。这个问题只支持 configurable operation-stack folding, 不支持完整生态兼容或自动恢复所有 latent intent。Prompt、tool、process 和 syscall 可以是 operation records 或 fields, session/span 是字段、容器或 baseline shape, 不是新的 profiler 抽象。

核心 14 数据集产生 42,590 个 operations 和 1,497 个 unique stacks, compression ratio 为 28.45。加入 ScaleCUA 补充样本后, 总量为 47,590 个 operations 和 1,611 个 unique stacks, compression ratio 为 29.541。这些 operations 覆盖 web、mobile、desktop、software、API、tool-agent-user dialogue、expert failure labels、safety labels、human grouped-action labels 和 desktop step-quality labels。所有这些标签都以 operation fields 形式进入同一 pipeline。没有任何数据集需要新增 prompt、session、GUI、safety 或 quality-specific profiler object。

E1 支持 C1 的机制性版本, 因为 agentpprof 可以把异构 agent 轨迹统一为 operations, 并用用户指定 stack 折叠。这个结果不说明所有 agent 数据都容易转换。适合该方法的轨迹通常同时具备有序 step 或 turn、稳定 session/task id、动作或阶段标签, 以及 outcome、quality、group 或 safety 等较高层 oracle。只有静态截图 grounding、缺少顺序、缺少 action label、强依赖大图片归档或 gated access 的数据源更适合作为后续扩展, 而不是当前主证据。

表 3 给出 E1 的证据路径, 后文围绕这些比较解释机制和边界, 而不是把来源核对写成单独实验。

递归 stack-depth sweep 说明固定 session 边界会迅速碎片化同一批 operations。主结果在同一批 13,265 个 operations 上扫过 8 个 stack depths。只保留 dataset 时有 9 个 stacks, 加入 task 后为 11 个, phase depth 为 57 个, tool/semantic depth 为 226 个, action depth 为 455 个。加入固定 session 后, unique stacks 膨胀到 3,757 个。相对于 dataset depth, 固定 session 的 unique stacks 增加 417.4 倍。这个结果直接反驳把 session 或 prompt 作为默认 profiling 边界的设计。Session 是有用 drilldown field, 但如果默认写死在 stack 里, 它会把可聚合的行为切碎。

早期 3,797-operation set 上也出现同样信号。Flat/mapped stack 为 103 个 unique stacks, 而固定 demo/session stack 为 1,083 个 unique stacks。这说明固定边界会带来约 10.5 倍碎片化, 而 mapped operation stack 可以保留聚合同时增加语义深度。因此论文中的 novelty 不是“画 flamegraph”, 而是把 agent profiling 的边界选择变成 operation-stack query。

Profile spec 把这个 query 自由度变成可复现实验配置。同一个 AgentNet spec 引用相同 operation JSONL 和 op-map 文件, 默认产生 608 个 diagnostic stacks。命令行只覆盖 --stack 和输出路径后, 样本数保持 16,741 不变, unique stacks 降到 83。这说明 stack 深度是可审计、可覆盖的查询参数, 而不是写死在 prompt/session 或某个可视化中的层级。

这种可配置性不只改变图的形状, 也改变可验证的问题。Depth sweep、stack tree、transition/top-field、quality、grouped-boundary 和 history-depth views 都服务同一个机制问题, 即 profiler 是否可以把边界选择延后到查询时, 而不是在采集时固定为 prompt、session 或 span tree。当前证据支持三条带范围的结论。第一, 异构轨迹和本地 session

表 2: 四个研究问题及其证据边界。每行给出 workload、oracle、主要证据和该证据支持的结论。

研究问题	Workload / oracle	主要证据	支持的结论
RQ1/E1: 泛化覆盖、递归折叠和字段派生	15 public labeled trace families / 47,590 operations, plus local-session and standard-trace exchange fixtures. OSWorld-Human and AgentNet provide the strongest boundary and step-quality oracles.	Core sources contain 42,590 operations; Scale-CUA brings the coverage sample to 47,590; the same operations fold from 9 dataset stacks to 57 phase, 226 semantic, 455 action, or 3,757 fixed-session stacks. AgentNet profile-spec override changes 608 stacks to 83 without changing input operations. Automatic operation-stack induction produces variable-depth stacks on 4/6 hidden-label tasks and stops on 2/6 when visible evidence has no material split. Held-out mapping improves compression from 14.049 to 19.091; OSWorld-Human learned boundary F1 reaches 0.7735; boundary backends beat the best simple baseline on 4/5 rows.	Heterogeneous traces enter one operation layer, and stack depth is a recursive query over fields. Mapping, tagging, and boundary backends derive fields before folding; they do not add prompt/session/tool/safety-specific profiler abstractions or automatic intent boundary discovery.
RQ2/E2: hidden-label 定位和排序	Six failure, safety, quality, and boundary tasks over AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human; 34,539 operations / 3,699 positives.	The benchmark scores 144 view/ranker policies. Query-aware operation-stack top-5 groups inspect median 0.0937 work versus 1.0 for flat, improve top-5 recall over fixed-session on 5/6 tasks, and reduce median groups from 285.0 to 157.5. Work-budget, fragmentation, fixed-recall, sequence-scope, oracle-depth, uncertainty, negative-control 和 multi-metric analyses test robustness rather than define separate experiments.	Operation-stack profiling is faithful to dataset-provided hidden labels under this benchmark and gives a better inspection-work/median-fragmentation Pareto point than flat or fixed-session alone. It is not a universal detector, a single best hierarchy, or a work-dominant replacement for drill-down views.
RQ3/E3: 机制隔离和可操作性	The same six labeled tasks, plus held-out OSWorld-Human boundary-backend operations.	Operation-level rank features and ablations expose useful and misleading fields; profile-spec patches improve 5/6 tasks with median AP delta 0.0376 and lift delta 0.5750. Induced depth sweep finds best query-aware median AP at depth 3 and lowest median top-5 work at depth 5. Learned-boundary fields improve OSWorld-Human AP to 0.2583 and reduce groups to 74, but increase top-5 work and first-positive work.	The profiler produces profile-configuration actionability: which operation fields, operation-stack depths, mappings, rankers, lenses, and drilldown views to tune. It keeps automatic action selection, automatic patch selection, and automatic boundary discovery out of scope.
RQ4/E4: 可重放性、离线成本和主张边界	76 operation-JSONL profile 配置, 以及 local-session, agent-trace 和 standard-trace input replay checks.	Deterministic replay 覆盖 152 次 profiler 执行。Semantic determinism 在两种模式下均为 76/76. Deterministic output 将 raw-byte determinism 提升到 76/76, median runtime 为 1.601s, p95 为 2.767s. Source selection, regex tags, trace inputs 和 effective standard-trace args 都通过同一 operation-stack path replay.	The offline profiler path is replayable and cheap enough for artifact evaluation. This result does not measure live eBPF overhead, human productivity, or full trace-ecosystem compatibility.

表 3: E1 主证据表: 同一个 operation layer 可以被递归折叠、mapping、tagging 和 boundary scoring, 而不引入 prompt/session/tool-specific profiler object。

Evaluation question	Workload/oracle	Main evidence	Counterpoint / scope
异构 operation layer	15 个 public labeled trace families / 47,590 operations	Web, mobile, desktop, software, API, dialogue, failure, safety, human-boundary 和 step-quality labels 都作为 operation fields 进入同一 pipeline。	这是 lightweight sampling; image-heavy 或受限访问的完整 benchmarks 仍是 out of scope。
递归 stack choice	同一批 13,265 operations, 加 AgentNet profile-spec override	9 个 dataset stacks, 57 个 phase stacks, 226 个 tool/semantic stacks, 455 个 action stacks 和 3,757 个 fixed-session stacks; AgentNet 在不改变 16,741 个 input operations 的情况下从 608 个 stacks 改成 83 个 stacks。	Fixed sessions 可以作为 drilldown fields, 但 hard-coded 到 stack 里会造成 fragmentation。
先派生 fields 再折叠	Held-out mapping 和 boundary-backend rows	Mapping 把 compression 从 14.049 提到 19.091; leave-dataset-out 在 6/9 个数据集上减少 stacks; boundary backends 在 4/5 rows 上优于 best simple baseline。	Mapping 会把 action-to-phase V-measure 从 0.9343 降到 0.7416; looping 由 repeat_signal_change 解释。
Human-boundary folding	OSWorld-Human exact-aligned tasks	4,011 operations / 2,075 human groups; grouped-depth stacks 把 482 个 action stacks 降到 106 个; group-pattern boundary F1 为 0.627, precision 为 1.0。	这是带范围的人类分组对齐结果, 不是自动恢复所有 latent intent boundary。

可以先进入同一 operation layer, 再用用户选择的 stack 折叠。第二, recursive stacks 能表达 dataset、task、phase、tool、action、human-group、safety 和 quality-label 等多种深度, 但不能声称恢复所有真实意图边界。第三, field derivation 是可扩展接口, 因为 mapping 和 supervised backend 只写入 operation fields, profiler 仍按 operation stack 折叠。这些机制结果支撑本文的新意, 但 user-utility、通用 boundary detector 和完整 trace-platform compatibility 仍是后续工作。

Rust operation-stack induction 进一步检验“不是用户给固定 stack 字段链”的实现路径。在 AgentRewardBench looping slice 上, agentpprof 用 `--induce-operation-stack` 从可见 evidence、semantic shift、changed-field density 和 query hint 给相邻边界打分, 并输出 induced operation-only stacks。在六个 hidden-label tasks 上, 同一个实现作为普通 profiler view 运行时, 4/6 tasks 产生 variable-depth stacks。2/6 AgentNet quality tasks 因可见 evidence 没有 material split 而停在一个 induced segment。其中 AgentRewardBench 和 OSWorld-

Human 的 stacks 覆盖深度 2/3/4, SATraj 同时保留深度 1/2/3/4。这个自动 view 的 median groups 为 12.0, 远低于 fixed-session 的 285.0。它的 median top-5 work 为 0.653, 低于 flat summary 的 1.0。同时它的 median AP 为 0.2762, 低于 hand-configured operation-stack query-aware 的 0.3116。同一 induced view 的 depth sweep 进一步说明深度是 profile 配置。Query-aware median AP 在 depth 3 最高, 为 0.2865, median top-5 work 在 depth 5 最低, 为 0.4727。有 material split 的四个任务 AP-best depth 分散在 2、3、4 和 5。这说明同一个 prompt/action 序列可以被自动折叠成参差不齐的 operation stack, 但它支持的是 configurable recursive folding 和 depth tuning, 不是取代 task-specific profile specs, 也不是自动发现所有 intent boundary。表 4 汇总这组三层证据。机制形成、hidden-label localization 消融和深度调优分别支撑不同主张, 并各自保留不支持的边界。

Field derivation 和 boundary probes 检验字段派生是否能在合适标签家族上迁移。这些 probes 支持的是 configurable deterministic/supervised field derivation, 而不是

表 4: 自动 operation-stack induction 的 claim-facing 证据。该表把递归形成、hidden-label 定位消融和深度调优连到同一 profiling claim; hidden labels 只在 profiling 后评分。

证据块	读者问题	主要数字	支持的结论 / 边界
E1 recursive formation	profiler 能否在没有用户指定 field chain 的情况下形成递归 operation stack?	729 个 operations; 15 个 induced stacks; 深度直方图 2:1/3:1/4:13; session-as-evidence view 有 16 个 stacks。	可见边界证据足以诱导参差递归的 operation-only stacks, session 只是可选 evidence field。不能声称自动发现所有 intent boundaries。
E2 localization ablation	induced stacks 能否作为真实 hidden-label tasks 上的可见 profiler view?	4/6 tasks 形成 variable depth, 2/6 tasks 在 material stop 停止; AP 0.2762 vs hand-configured 0.3116; work@5 0.653 vs flat 1; groups 12 vs fixed-session 285。	induction 降低 flat inspection work 和 fixed-session fragmentation, 但 AP 仍弱于 hand-configured specs。不能把它写成 task-specific profile specs 的替代品。
E3 depth actionability	induced-stack depth 是否是实际可调的 profiling surface?	query-aware median AP 在 depth 3 最高 (0.2865); median work@5 在 depth 5 最低 (0.4727); material-split AP-best depths 覆盖 2, 3, 4, 5。	不同目标偏好不同递归深度, 因此 depth 是 profile-configuration knob。不能声称自动 depth selector 或 analyst-productivity 改善。

通用无监督边界发现。Held-out mapping rules 在 3,990 个 operations 上把 compression 从无 mapping baseline 的 14.049 提高到 19.091, held-out stacks 从 284 降到 209, 并把 task/dataset V-measure 从 0.837 提高到 0.853。Phase/action boundary F1 为 0.777、precision 为 1.0, 说明当前 mapping 保守。它不会制造 false positive phase changes, 但会合并一些 fine-grained action changes。Leave-dataset-out mapping 在修正 API/tool phase precedence 后, 在 9 个 held-out datasets 中的 6 个减少 unique stacks, 0 个出现 negative stack-reduction regression, weighted stack reduction 为每 1,000 operations 1162.005。这些结果说明 mapping 不是单个数据集的 ad hoc pretty printer, 但它仍是 deterministic taxonomy-seeded field derivation, 不是无监督边界发现。

Boundary backend 提供更强但仍带范围的 field derivation 证据。在 OSWorld-Human exact-aligned sessions 上, held-out adjacent-pair backend 使用非 oracle operation fields 训练, 明确排除 human_group、group_index、group_position、group_size、group_pattern 和 group_alignment。它在 held-out pairs 上达到 precision 0.7400、recall 0.8102、F1 0.7735, 高于 phase-change baseline 0.2919、action-change baseline 0.5142 和 conservative group_pattern reference 0.5810。预测字段写回 learned_group_pattern 和 learned_group_position 后, Rust agentpprof 在 1,132 个 held-out operations 上折叠得到 74 个 stacks。这支持 supervised boundary fields 可以作为普通 operation fields 被 folding 使用, 不支持新增第三个 profiler object。

Field-derivation analysis 同时保留反例。Mapping 提高 compression, 但把 action-to-phase V-measure 从 0.9343 降到 0.7416, 说明它是聚合机制而不是细粒度边界 oracle。跨标签家族的 boundary suitability check 中, learned boundary backends 在 4/5 rows 上胜过 best simple baseline。可训练 rows 的 held-out F1 分别是 OSWorld-Human 0.6916、AgentNet step-correct 0.3197、AgentNet step-redundant 0.3361 和 AgentRewardBench looping 0.7833。AgentNet 两个 quality-state boundaries 优于 always-boundary baselines (0.2155 和 0.2645), 但仍有 precision 和 calibration

caveat。AgentRewardBench looping 虽然 learned F1 达到 0.7833, 却被 repeat_signal_change baseline 的 1.0 F1 完全解释。因此 E1 的结论是, mapping、tagging/rank features、profile specs 和 supervised boundary backends 都只是写入 operation fields, 然后由 operation stack 折叠。它们支持 first-class field derivation 和 suitability checks, 不支持 automatic intent boundary discovery。Suitability analysis 把这个扩展策略写成更明确的配置决策。Deterministic mapping 适合 compression, 但会粗化 action boundary。Rank features 需要 ablation evidence, 因为真实任务里既有 critical feature rows, 也有 misleading feature rows。Boundary-derived fields 必须先过 family-specific suitability。当前 boundary-family 决策包含 1 个 accept、3 个 caution 和 1 个 reject, 因此 field derivation 是有条件的 profile-configuration loop, 不是 universal automatic selector。

当前最强的 grouped-boundary oracle 来自 OSWorld-Human。转换器覆盖 369 个 human desktop tasks 和 6,010 个 single-action operations。只有 grouped-action 序列与 single-action 序列 exact-aligned 的 320 个 tasks 被写入 human_group、group_pattern 和 group_position oracle fields。这些 exact-aligned tasks 含 4,011 个 operations 和 2,075 个 session-local groups。31 个 content-mismatch tasks 和 18 个 length-mismatch tasks 保留用于 action profiling, 但不被当作 gold group oracle。在同一批 exact-aligned operations 上, action-depth stack 有 482 个 unique stacks, grouped-depth stack 降到 106 个。保守 group_pattern 对人工 human_group boundary 的 F1 为 0.627, precision 为 1.0, recall 为 0.456。这不是完美边界 detector, 但它给出有意义结论: operation stack 可以在同一条单动作序列上选择 action-depth 或 group-depth, 并以人工 grouped-action 标签作为 oracle 验证。Prompt 或 session 没有参与这个抽象。

5.2 RQ2/E2: Hidden-label localization 和 ranking

RQ2 检验 operation-stack profiling 能否在真实 labeled traces 上完成 faithful localization 和 ranking。我们把

6 个 oracle-backed tasks 中的 34,539 个 operations 和 3,699 个 hidden positives 作为评分对象, 并比较 flat summary、fixed-session drilldown、dataset-native hierarchy、raw-action stack、operation-stack width、operation-stack query-aware、label-drilldown 和 oracle upper bound。评分指标包括 precision@k、recall、F1、AUPRC-style AP、nDCG、recall/F1@work budget、work-to-first-positive 和 group count。Flat 和 dataset-native 可以赢 broad-recall 或 nDCG 目标, 因此主张必须是 Pareto tradeoff, 而不是 metric dominance。这一节的核心问题是 hot stacks 和 top-ranked groups 是否真的对应 hidden positives, 并且是否能相对 flat summary 降低 inspection work、相对 fixed-session drilldown proxy 改善 localization-budget/fragmentation tradeoff。

E2 把每个 profile group 当作 inspection target。Ranked targets 如果能用比 flat summary 更少的 work 找到 hidden positives, 并比 fixed-session trace-tree proxy 更少碎片化, 就支持本文的 profiling 主张。成功条件不是单一指标第一。Operation stack 需要降低 flat inspection work, 在有意义的 fixed-session localization 或 fragmentation objective 上改善, 同时保留 fixed-session early-hit 反例。这个主实验不重新定义 agent benchmark, 而是把真实标注轨迹中的 failure、safety、quality 和 human-boundary labels 当作 profiler localization oracle。6 个任务覆盖 AgentRewardBench looping/side-effect、SATraj unsafe、AgentNet incorrect/redundant steps 和 OSWorld-Human group starts, 总计 34,539 个 operations、3,699 个 positives。E2 scoring pipeline 把每个 view/ranker 的 hot groups 当作 ranked localization result, 用 hidden labels 计算 precision@k、recall@budget、F1、AUPRC-style AP、nDCG 和 work-to-first-positive。比较对象包括 flat、fixed-session drilldown、dataset-native hierarchy、raw action stack、operation-stack、label-drilldown oracle view, 以及 width、visible-risk、query-aware 和 oracle-upper-bound rankers。非 oracle policies 只使用 visible operation fields, label-drilldown 和 oracle-upper-bound 只作为上界。表 5 给出主要 policies 的 median 结果。Operation-stack query-aware 在 top-5 groups 中只检查 0.0937 operation fraction, 而 flat summary 必须检查 1.0。相对 fixed-session query-aware drilldown, operation-stack top-5 recall 在 5/6 tasks 上更高, 并把 median group count 从 285.0 降到 157.5。这支持的是 inspection-work/fragmentation Pareto tradeoff, 而不是 metric dominance。Dataset-native hierarchy 的 median top-5 recall 达到 0.8665 但 work 为 0.8539, fixed-session 的 work-to-first-positive 只有 0.0044 但 top-5 recall 只有 0.0233。

Failure interpretation 和 baseline scope 也是结果的一部分。Fixed-session drilldown 是当前已评估的 trace-tree-shaped baseline。这个 baseline 不等价于完整导入 OpenTelemetry、Phoenix、LangSmith、Langfuse 或 Peretto ecosystem traces。在同一批 scored rows 上, operation-stack query-aware 相对 fixed-session 在 5/6 tasks 上提高 top-5 recall 和 F1, 在 4/6 tasks 上提高 budget-30 recall, 并在 4/6 tasks 上减少 group count。同时 fixed-session 在 4/6 tasks 上保留更低 top-5 work 和 first-positive work。Depth-aware slice 细化同一主张。Operation-stack rankings 在 20/24 task-depth rows 上超过 fixed-session budget-30 positive-unit recall, 并在 22/24 rows 上用更少 groups 到达 50% positive units。如果这些反例从论文里

消失, 主张就会被写过头。保留它们后, E2 支持的是更好的 localization-budget/fragmentation tradeoff, 不是对所有 trace-tree-shaped objectives 的 dominance。Automatic induced operation stacks 给出同一 tradeoff 的机制侧证据。它们在不要用户给固定 stack 字段链的情况下, 用可见 boundary evidence 产生 variable-depth groups 或在证据不足时停止, 并在同一 hidden-label scoring pipeline 下取得 median top-5 work 0.653 和 median groups 12.0。同一 induced path 上改变 depth cap 会改变 profile 表面: depth 3 给出最高 query-aware median AP, depth 5 给出最低 median top-5 work, 且有 material split 的四个任务 AP-best depth 与 work-best depth 都不同。这个结果比 flat 更省检查、比 fixed-session 更少碎片化, 但 AP 仍低于 hand-configured operation-stack, 所以 E2 主结论仍以 query-aware operation stack 为主。Case views 只解释 localization result 的可复查性。这些 examples 只使用可见 operation fields, 再用隐藏标签事后评分, 因此可以展示 top-ranked operation-stack groups 为什么命中或误导, 同时保留 flat full-recall 与 fixed-session early-hit 反例。主准确性主张仍然来自统一的 hidden-label scoring, 而不是来自 case 展示本身。

同一批 scored groups 也可以组织成 flamegraph 之外的分析视图。主文只保留一个代表性图, 把 E2 的 localization/work/fragmentation tradeoff 和 E3 的 profile-configuration knobs 放在同一分析视图中。其他任务级 views 作为补充材料保留, 用来解释主实验而不是制造新的主实验。

任务级 views 把主实验的 aggregate result 落到具体诊断结论上。它们显示 6/6 tasks 相比 flat summary 改善 AP 和 top-five inspected work, 5/6 tasks 相比 fixed-session drilldown 改善 top-five recall, 4/6 tasks 降低 fixed-session group fragmentation, 并且 6/6 tasks 都有 actionable configuration result, 即 profile-spec patch 或 boundary-field repair。这些 views 也保留负证据: fixed-session 在若干任务上仍然是 first-positive 或 group-count counterpoint, OSWorld-Human 需要 boundary-derived fields, 而不是 visible phase/action ranking alone。

E2 的 robustness evidence 只检验 Pareto 结论是否稳健。Bootstrap 和 label-permutation analyses 显示, operation-stack query-aware 的 AP、top-5 work、budget recall 和 group-count deltas 在 6/6 tasks 上方向稳定。AP 在 6/6 tasks 上超过 95% prevalence/group-size null, 30% budget recall 在 5/6 tasks 上超过 null。Work-budget、fragmentation 和 oracle-depth analyses 再把收益拆到检查成本上。30% work budget 下 median recall 为 0.3900, 高于 fixed-session 的 0.3559 和 flat 的 0.0000。相同 budget 下它在 5/6 tasks 上检查更少 groups, 同时保留 fixed-session first-positive-work 反例。在 24 个 task-depth rows 中, 它有 20 个提高 fixed-session budget-30 positive-unit recall, 并在 22 个 rows 用更少 groups 达到 50% positive units。这些 analyses 合起来支持 faithful localization surface 和 inspection-work/fragmentation tradeoff。它们不支持 single best hierarchy、human productivity 或对所有 trace-tree-shaped objectives 的 dominance。

E2 的带范围结论是 operation-stack profiling 在真实标注轨迹上能把 hot groups 排成对 hidden positives 有信号的 localization surface, 相比 flat summary 明显减少 inspection work, 相比 fixed-session drilldown 通常改

表 5: E2 hidden-label localization benchmark. Hidden 表示 policy 是否读取 oracle label; WTFP 是 work-to-first-positive. 非 oracle policies 只读 visible operation fields, hidden labels 只用于 scoring.

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	no	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	no	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	no	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	no	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	no	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	no	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	yes	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	yes	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

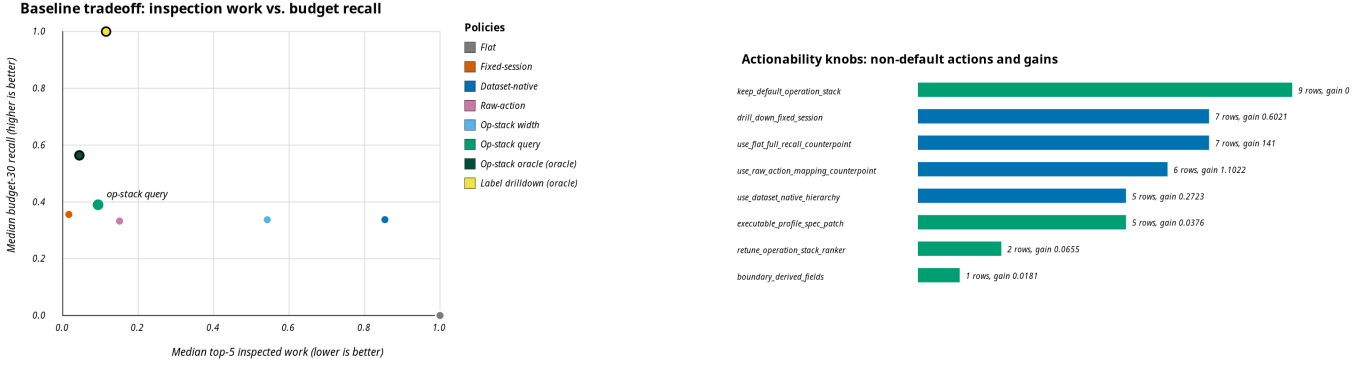


图 1: 两个非 flamegraph 分析视图。左图把 RQ2/E2 的核心 tradeoff 放在同一坐标系中, operation-stack query-aware 位于 flat summary 的低 work 区域, 同时保留 fixed-session 和 oracle 上界反例。右图把 RQ3/E3 的 actionability 展开为 profile-configuration knobs, 显示多数 objective rows 需要改变 view、ranker、profile spec 或 boundary-derived fields, 而不是使用一个固定默认视图。

善 median fragmentation tradeoff 或提高同预算 recall。它不能写成 automatic detector、single best hierarchy、human/agent analyst productivity, 或对真实 OpenTelemetry/Phoenix/LangSmith/Perfetto span tree 的完整兼容和支配。Work-budget、fragmentation、fixed-recall、sequence-scope、oracle-depth、uncertainty、negative-control 和 multi-metric analyses 共同约束同一个 E2 主实验的适用范围。

5.3 RQ3/E3: 机制隔离和 actionability

RQ3 检验 profiler 是否能够通过 operation fields、operation-stack depth、mapping/tagging rules、ranking policies、profile specs 和 boundary-derived operation fields 暴露 actionable optimization knobs。Hidden labels 仍然只在 profiling 之后用于 scoring, boundary-derived fields 还用 held-out OSWorld-Human boundary positives 评分。我们比较 default semantic-width specs、patched specs、visible feature rankers、transfer policies、learned-boundary stacks、fixed-session 和 semantic-width stacks, 并报告 accepted patches、AP delta、top-5 lift delta、first-positive-work delta、group reduction 和 top-5 work counterpoint。OSWorld-Human 需要 boundary-derived fields, 这说明 RQ3 评估的是可操作机制证据, 而不是自动边界发现或自动 patch 选择。这一节的问题是 localization gain 来自哪些机制, 以及 profile-guided 的 operation fields、operation-stack depth、rank features、mappings 或 boundary-derived fields 能否在不把 hidden labels 泄漏进 profiler input 的前提下改善输出。Rank-feature、mapping、stack-depth、boundary-field、profile-spec patch、case view、diagnostic lens 和 transfer counterpoint 共同支撑同一个 RQ3 主张。

E3 的机制问题是 E2 的 ranked groups 能不能解释应该怎样改 profiler 配置。Hidden labels 只在 profiling 之后用于 scoring。Visible inputs 仍然是 operation fields、operation-stack depth、mapping/tagging rules、rank features、profile specs 和 boundary-derived operation fields。Actionable diagnosis 需要每个任务指向具体的 operation

field、stack depth、ranker、mapping、profile-spec 或 drill-down change, 并且 executable patches 能改善 scored output。这个证据仍不能写成 automatic selector。

表 6 是 E3 的主显示, 它把 task-level localization signal 转成 diagnosis table。SATraj unsafe 说明 environment、phase、action 和 risky/write-action ranking 对安全问题有效。Looping 说明 repeat_signal 有用但需要 prevalence-aware ranking, side-effect 指向更深的 write/input mapping。AgentNet step-quality 需要 action-family 定位后再进入 fixed-session examples, OSWorld-Human 则需要 boundary-derived fields。因此 actionability 的第一层结论是 profiler 指出该调 operation field、stack depth、mapping、ranker、profile spec、boundary field 还是 drilldown, 而不是给出唯一默认 view。Task-level verdict synthesis 给出任务级结论。6/6 任务都是支持但有反例, 不是无条件胜出。其中 5/6 任务有 accepted executable profile-spec patch。剩下的 OSWorld-Human row 指向 boundary-derived fields, 并保留 top-five 与 first-positive work 反例。这 6 行进一步收敛成三类可复用优化模式。Safety 和 side-effect 任务指向 field 与 ranker 修改, 因为 environment、phase/action 和 write/input fields 会把相关 positives 推到更小的 inspected region。Step-quality 任务指向 drilldown workflow, 因为 desktop phase、repeat 和 action fields 先定位可疑区域, 再由 fixed-session examples 解释单条 trace。Human-boundary 任务指向 representation repair, 因为 action-depth stack 不够, boundary-derived operation fields 能提高 AP 并减少 groups, 但仍保留 inspection-work 反例。这些模式给出的是 profile inputs、operation fields、stack depth 和 rankers 的配置建议, 不是 profiler 自动预测出的标签。Field-suitability decisions 进一步说明这些配置建议何时可用。Deterministic mapping、stack depth 和 predicates、operation-level rank features、supervised boundary fields、boundary-profile repair 分别对应不同的 accept/caution/reject 条件。因此 E3

的 actionability 是 profiler 指出该调哪个 operation-field 或 operation-stack view, 而不是自动选择唯一正确视图或自动发现所有边界。Automatic operation-stack induction 也落在这个 actionability 边界内。它证明 profiler 可以从可见字段递归诱导 operation-only stack, 并把深度参差的 stack 当作可打分 view, 但它的 AP 反例说明 profile-specific stack 和 ranker 仍然需要按任务调优。Depth sensitivity 把这点变成可操作结论: 同一 induced profiler view 在 depth 3 取得最高 query-aware median AP, 在 depth 5 取得最低 median top-5 work, 而有 material split 的 task-level AP-best depth 跨 2、3、4、5 分布。因此 E3 的建议是调 operation-stack depth 和 ranker objective, 而不是把 `analysis_task, phase, action, status` 当成固定 ontology。

机制消融说明这些诊断来自可复现的 profiler 配置。Stack-text rank rules 和 rank mode 可以从 profile spec 复现, 但它们对 safety/side-effect 过粗。Operation-level rank features 在 folding 前匹配 visible mapped operation tokens, semantic stack 的 feature ranking 在 AP 上赢 5/6 tasks, top-5 lift 赢 4/6, first-positive work 赢 5/6, coarse depth 在 4/6 tasks 上改善 AP 并减少 groups。Leave-one-feature 和 robustness analyses 说明哪些字段有用或有害。repeat_signal、write-action 和 status=success 在不同 family 关键, SATraj loop-like 或 OSWorld-Human input-phase features 会误导。Global/equal feature banks 常能保留收益, guided repair 在 2/3 misleading-feature cases 上改善 AP。这些 repair 使用离线评分反馈, 因此支持 mechanism isolation。它们不是自动 learned ranker, 也不支持 automatic learned ranker。

View、depth 和 budget 审计把机制结果放回 profiling tradeoff。Best AP 和 best top-5 F1 分散在 operation-stack、fixed-session、dataset-native、raw-action 和 flat views。Leave-task source selection 不能稳定选中 best view。但 operation-stack query-aware 在 6/6 tasks 上进入 Pareto frontier, 相对 flat 在 6/6 tasks 降低 top-5 work, 相对 fixed-session 在 5/6 tasks 提高 top-5 recall, 并在 5/6 tasks 用更少 groups 达到 50% positives。25% recall target 下它覆盖 6/6 tasks, median work 为 0.2000 而 flat 为 1.0000, median groups 为 16.0 而 fixed-session 为 50.0。50% recall 和 first-positive work 仍保留 fixed-session、dataset-native 或 raw-action counterpoints。Sequence-scope 和 oracle-depth 审计进一步说明同一个检查成本结论。30% work budget 下 positive-session recall 为 0.4669, 高于 fixed-session 的 0.3230。24 个 oracle-depth rows 上 budget-30 positive-unit recall median 为 0.4342, 并在 20/24 rows 提高 fixed-session unit recall, 22/24 rows 用更少 groups 达到 50% positive units。Positive-run proxy units 是 episode-derived proxy units, 不是 human intent。positive-session-without-unit fixed-session depth-gap counterpoint 仍然保留, 所以这只是 depth-aware triage, 不是自动发现所有 latent subtask 或 intent boundary。多指标审计也保留反例: 30 个 baseline-metric comparisons 支持 operation-stack query-aware, 16 个是明确 counterpoints, nDCG 场景下 flat 或 dataset-native 可能胜出。

Actionability view 把这些比较变成可以检查的优化建议。36/36 objective rows 都有具体 profile-configuration action, 但 27/36 best visible policies 不是默认 operation-stack query-aware。Transfer miss 多数对应 view 或 ranker

变化, 而不是不可解释错误。同一证据可以从 diagnostic lenses、case examples 和 baseline contrasts 三个角度检查。6 个 lenses 覆盖 36 objectives, top-5 case groups 在 6/6 tasks 命中 positive, top-1 在 5/6 命中, median top-5 lift 为 1.6508 且 work 为 0.0937。Operation-stack 在 6/6 tasks 胜过 flat top-5 work、在 5/6 胜过 fixed-session top-5 recall, 同时保留 fixed-session first-positive 和 flat full-recall 反例。Visible counterfactual 和 held-out transfer results 把行动建议与目标标签泄漏分开。36/36 best rows 是 visible non-oracle, 27/36 需要 non-default action, 25/36 需要 view change。Held-out action transfer 在 35/60 aligned decisions 上 within tolerance, 但 exact action-class transfer 只有 7/60, 在 non-default target action 上只有 2/42。因此这里的主张是可调诊断界面, 不是 label-free automatic selector。

Executable profile-spec patches 关闭一个从诊断到可执行配置的闭环。Default 与 patched profile specs 在同一 visible operation input 上实际运行 Rust agentpprof --profile-spec, hidden labels 只在 profiler 输出后评分。5/6 patches 同时改善 AP、top-5 lift 和 first-positive work, median AP delta 为 0.0376, median top-5 lift delta 为 0.5750。这里的 failure interpretation 同样重要: 唯一 reject 是 OSWorld-Human, 说明 visible phase/action rank features 不足以处理 human-boundary 目标。Held-out boundary-derived fields 随后作为普通 operation fields 折叠, AP 从 semantic-width 的 0.2402 提升到 0.2583, groups 从 108 降到 74。反例是 top-5 work 和 first-positive work 变差。这些 patch 和 boundary-field results 支持 boundary-derived fields 是 actionable operation-field knob, 不支持 automatic boundary discovery、automatic patch selector 或第五个核心实验。

5.4 RQ4/E4: 可重放性、离线成本和主张边界

RQ4 问复现者能否重放支撑 E1–E3 的离线 profiler 输出, 并看清哪些主张仍未被测量。我们把 76 个 profile 配置和对应 operation inputs 当作复现单位, 同时检查本地 session、agent-session trace 和 standard trace 这些输入来源是否都进入同一 folding path。我们比较 default output 与 deterministic-output replay, 比较 semantic profile hashes 与 raw-byte profile hashes, 并验证声明的 input paths 是否实际进入 operation-stack folding path。主要指标包括 spec pass rate、profiler invocation 数量、median/p95 runtime、sample equality、stack equality、raw-byte output equality 和 input-source coverage。这个问题只覆盖离线路径复现, 不覆盖 live eBPF overhead、human utility、complete ecosystem compatibility 或 automatic universal selection。

本节只报告可重放性和离线成本。在同一批 operation JSONL inputs 上, agentpprof 重复执行 76 个 profile 配置, 共 152 次 profiler invocations。默认模式下所有 specs 在重复运行中保持 semantic profile content、samples 和 unique stacks 稳定, median 每个 spec 运行时间为 1.581s, p95 为 2.719s。Raw-byte determinism 只有 4/76, 因为输出带生成时间戳。Deterministic replay mode 固定 JSON generated_at 和 pprof profile time 后, 同一 76 个 specs 的 semantic 与 raw-byte determinism 均为 76/76, median 每个 spec 运行时间为 1.601s, p95 为 2.767s。这些结果说明复现者可以重放离线 profile content, 并在需要时得到 byte-stable profile files。它们不等价于 live capture overhead, 也不形成另一个经验性 profiling 实验。

输入来源 replay 说明 profile spec 是真正的数

表 6: E3 diagnosis/actionability summary. 每行把 hidden-label localization 信号、可执行 profile 配置动作和带范围的 verdict/counterpoint 放在一起, 展示 profiler 如何指向可调的 operation-field、stack-depth、ranker 或 boundary-field 决策。

Task	Localization signal	Profile action	Verdict / counterpoint
Looping	Work@5 0.4938, R@5 0.6508, patch AP +0.1155	保留 repeat_signal , 但加入 prevalence-aware ranking, 因为 positives 很常见。	支持但有反例: raw-action first-positive work 更低。
Side-effect	Work@5 0.1454, R@5 0.1139, patch AP +0.0591	提高 write/input 权重, 或在 ranking 前使用更深的 side-effect mapping。	支持但有反例: fixed-session groups 更少; raw-action top-five lift 更好。
Unsafe operation	Work@5 0.0420, R@5 0.2621, patch AP +0.5081	使用 environment、phase 和 action operation fields, 并选择更深的 operation-stack view; 优先 risky environment 和 write actions。	支持但有反例: fixed-session first-positive work 更低。
Incorrect step	Work@5 0.0014, R@5 0.0034, patch AP +0.0160	使用 desktop environment、phase、repeat 和 action fields, 再进入 fixed-session examples。	支持但有反例: fixed-session 仍是 drill-down baseline。
Redundant step	Work@5 0.0089, R@5 0.0177, patch AP +0.0011	使用 desktop environment、phase、repeat 和 action fields, 再进入 fixed-session examples。	支持但有反例: fixed-session first-positive work 更低。
Group start	Work@5 0.4074, R@5 0.3874, boundary AP 0.2583	使用 group-depth 或 boundary-derived fields; action-depth alone 会 fragment starts。	支持但有反例: boundary fields 把 groups 降到 74, 但 top-five 和 first-positive work 变差。

表 7: E4 可重复性和成本主显示。Runtime 为 Rust binary 已存在后的每个 spec 秒数; semantic hash 去除 JSON generated_at 字段, raw-byte hash 单独报告。

Workload	Specs	Med. s	P95 s	Med. stacks
View specs	4	3.448	4.273	631.0
Rank-feature specs	12	1.539	2.692	41.0
Robustness specs	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

表 8: E4 deterministic-output 主显示, 比较同一 76 个 profile specs 的默认输出和 --deterministic-output replay。

Config	Mode	Sem.	Raw	Med. s	P95 s
Default	default	76/76	4/76	1.581	2.719
Deterministic	deterministic	76/76	76/76	1.601	2.767

据入口配置, 而不是只包了一层 output path. Rust profile-spec path 现在可以携带 session_files、trace_files、standard_trace_files、regex tag_rules、predicates、rank rules、stack overrides 和 include_standard_trace_args。回归测试把同一个 public Codex fixture 作为 local session 和 exported agent-session trace replay, 并把一个 generic Chrome Trace Event 中的非 AgentSight protocol arg 导入为 operation field 后折叠。这些输入在导入后都归一成普通 operations, 再由同一 operation-stack machinery 折叠。这个结果支持 trace exchange 的最低复现要求, 但不支持完整 trace-ecosystem compatibility 主张。表 7 和表 8 是 E4 的两个主显示。前者呈现 replayability/cost result, 后者呈现 deterministic-output repair。表中的 labels 只标识 profile-spec families, 不是新的 accuracy 实验。

除了两张主表, RQ4 还约束论文数字、数据来源选择和明确排除的主张。这些一致性约束只核对已经报告的 profile results 和当前论文文本, 不重新排名、不下载、不同步、不创建、不重标数据集、不重跑 profiler, 也不运行 human/agent analyst task。上一节的 actionability 结论依赖这些可复现 profile 配置和 operation inputs。本节不重新验证 fidelity/accuracy、actionability 或 baseline tradeoff, 只证明复现者能重放这些配置, 并核对正文数字、数据来源选择和主张边界。因此 RQ4/E4 只支持复现和主张边界, 不是人工 analyst 主观解释、live overhead 或新的 profiler accuracy 结果。

E4 的辅助材料只约束正文和可重放结果的一致性。它们把论文主张、数据来源选择、明确排除的结论和 operation/operation-stack 边界对齐到 replayed profiles。这些材料帮助读者判断论文是否把证据写过头, 但不增加 localization、actionability 或 overhead 主张。这些辅助视图可以支持 E1–E4, 但不会形成额外主实验。

6 数据源选择与扩展边界

当前数据选择遵循 oracle-first 原则。Operation-stack profiling 的主张需要有顺序、有动作、并且能隐藏评分标签的真实轨迹。规模更大的数据集如果缺少这些字段, 只能支撑覆盖范围, 不能支撑 localization accuracy。OSWorld-Human 提供 single-action 和 grouped-action, 因此最直接支撑递归边界验证。AgentNet 同时提供大规模人工桌面任务和 per-step correctness/redundancy, 因此支撑 step-quality profiling。AgentRewardBench 和 SATraj-OS 把评估目标从 action taxonomy 推到 failure、looping、安全和 side-effect diagnostics。GUI-Odyssey、tau-bench 和 Tool-Bench 则作为跨域覆盖来源, 说明 operation abstraction 可以覆盖移动 GUI、user-agent-tool dialogue 和 API/tool traces, 而不局限于桌面轨迹。

ScaleCUA 的作用是扩大 GUI history 覆盖, 而不是提供新的主证据。我们使用的 ScaleCUA stream sample 含 5,000 个 Ubuntu navigation rows, 覆盖 131 个 sessions, 最大 step 为 48。该子集的动作 taxonomy 主要是 click 和 terminate, 所以 phase/action 指标达到 1.0 并不表示强泛化。它更适合证明 multi-step GUI history context 可以被投影为 history_state 和 history_depth fields, 而不是证明复杂边界 detector。

7 相关工作

7.1 Profiling、Flamegraphs 和 Trace Analysis

传统 flamegraph 和 pprof 路线把运行时调用栈折叠成热点视图 [1,16,17]。pprof 不只是固定 call stack, 因为它支持 sample tags, 也支持用 tagroot/tagleaf 把 tag value 作为 pseudo stack frame 可视化 [17]。Perfetto 则代表系统 trace ingestion、SQL trace analysis、trace summary 和 derived events 的成熟方向 [18]。这些系统已经覆盖 query-time aggregation 和 trace analysis 的关键能力, 所以本文的新意必须落在 agent 语义对象与标注评估的组合上。本文复用 folded-stack 表示, 贡献来自 agent-operation record model、recursive multi-field operation-stack projection 和真实标注轨迹上的 hidden-label localization benchmark 的组合。Stack frame 来自 agent operation fields, 而不是语言运行时函数调用或通用 trace SQL schema。因此同一批 operations 可以被折叠为 task、phase、tool、action、quality 或 safety 视图, 也可以改成 fixed-session 或 flat baseline。区别不在图形样式, 而在 agent 语义对象、递归多字段 stack projection 和跨数据集 hidden-label scoring。

7.2 Agent Tracing 和 LLM Observability

OpenTelemetry GenAI 和 OpenInference 标准化 GenAI spans、LLM calls、agent reasoning steps、tool invocations、retrieval operations、MCP 和 provider-specific telemetry [2,11]。LangSmith、Langfuse、Phoenix 和 AgentOps 等系统进一步把 LLM call、tool call、latency、token、trace tree、session、observation、evaluation score、goal、plan 和 agent artifact 放进单次 workflow inspection 与生产监控 [12,13,14,15]。这些系统是本文最接近的同问题相关工作。本文因此把 trace/span/session 明确当作 exchange container 或 baseline shape，而不是加成第三个 profiler 抽象。E2 实际评估的是 fixed-session drilldown 这个 trace-tree-shaped baseline。真实 OpenTelemetry/OpenInference/Phoenix span-tree import 仍是后续互操作 baseline，不是当前已完成结果。本文关注跨轨迹 semantic profiling，即按 agent operation fields 做跨 session 聚合、排序和评分。系统先把异构标注轨迹或本地 agent 历史统一为 operations，再允许用户选择 stack 深度、mappings、tagging 和 ranker。这些系统构成最接近的基线威胁，本文定位为互补的 profiler abstraction，而不是替代现有 observability products。

7.3 Agent Failure 和 Span Localization

2026 年的 AgentRx、TELBench/DRIFT、Holistic Evaluation and Failure Diagnosis 和 AgentAtlas 是最接近的同问题威胁 [23–26]。AgentRx 发布带 critical failure step 和 failure category 的失败轨迹，并用 constraint checking 与 LLM judge 做 root-cause localization。TELBench/DRIFT 把 deep-research agent 轨迹转成 semantic spans，并标注 harmful error spans。Holistic Diagnosis 把 agent-level diagnosis 和 span-level evaluation 结合起来。AgentAtlas 则强调 outcome leaderboard 不能覆盖 trajectory quality 和 control-decision quality。这些工作已经覆盖 failure localization、trajectory diagnosis 和 span-level evaluation 的问题定义。本文的差异更窄，把 profile groups 当作 profiler 输出，在同一批 operations 上比较 flat、fixed-session drilldown、dataset-native、raw-action 和 operation-stack views，并用 AP、nDCG、inspection budget recall、work-to-first-positive、fragmentation 和 profile-configuration actionability 评分。AgentRx/TELBench-style traces 是后续 oracle/baseline 候选，不是当前已完成的 broader span-localization 证据。

7.4 Computer-use 和 Tool-use Benchmarks

WebLINX、Mind2Web、GUI-Odyssey、OSWorld/OSWorld-Human、OpenCUA/AgentNet、SATraj-OS、AgentRewardBench、ToolBench 和 tau-bench 为本文提供真实标注轨迹 [3–10,19–22]。本文不提出新 benchmark。贡献是把这些 benchmark 的动作、质量、安全和人工 group 标签统一为 operation fields，并用同一个 profiler 机制比较它们适合回答哪些 profiling 问题。这些 benchmark 提供证据来源和原生序列对照，不是被本文替代的对象。

7.5 新意边界

本文不把新意建立在“第一个 agent observability 系统”、“第一个 agent trajectory benchmark”或“第一个 folded-stack 可视化”上。本文当前可支持的新意是只用 operation 和 operation stack 两个 profiler 抽象，把 trace/session/span 作为输入容器或 baseline，在真实公开

标注轨迹上把 profile groups 当成 ranking/localization outputs，并用 hidden labels 计算 accuracy、work、fragmentation 和 actionability。更具体地说，贡献不只是 query-time aggregation，而是把 agent-operation record、recursive multi-field stack shape 和 hidden-label localization benchmark 绑在一起。同一批 agent operations 会因为 stack shape 不同形成不同的可定位、可排序、可检查 aggregation units。这不是穷尽式 2026 全景综述。它只界定当前主张的最近同问题威胁和已评估边界。

8 局限

当前证据支持的是有范围限定的 profiling 主张，而不是不加限定的 agent-analysis 主张。字段派生和边界恢复仍然是第一个限制。当前证据支持 deterministic mappings、label-derived rules 和部分 supervised boundary backends 能派生 stackable operation fields，但没有证明无监督或 LLM-backed intent detector 已经解决。这些结果也没有证明一个 backend 能跨所有标签家族工作。Free-form prompt labeling 和 LLM taggers 仍需要在同一 hidden-label localization protocol 下与 deterministic mapping 比较，才能成为本文主张的一部分。AgentRewardBench looping 被简单 repeat_signal_change baseline 完全解释，这说明每个 family 都需要 suitability、calibration 和 simple-baseline 检查。

E2 支持 profiler-localization 主张，但没有测量人类效用。E2 把 failure、safety、quality 和 human-boundary 问题转成 hidden-label ranking/localization benchmark，并证明 operation-stack profiling 相对 flat summary 和 fixed-session drilldown 有 accuracy/work/fragmentation trade-off。这个 tradeoff 是 task- 和 metric-specific 的。最佳 visible view 不由单一 hierarchy 垄断，source-task selector 不能稳定自动选中最优 view。20%/30% inspection budget 下 operation-stack query-aware 的 median recall 更高，但 fixed-session 仍保留 first-positive 低成本反例。Fixed-session fragmentation 与 operation-work 成本也必须分开解释。Operation-stack 支持更少 group fragmentation 和同预算更少 groups，但 first-positive/work 指标仍不由 operation-stack 支配。相关工作比较只把 closest-system grounding 转成基线威胁，不增加新的实证结果。可选 analyst-study protocol 已经覆盖 6 个 tasks、3 个 views、24 个 participant slots 和 144 个 trials，并把 study materials 与 hidden answer key 分离。该 protocol 可用于后续验证开发者或 agent analyst 是否更快或更准。本文当前只评价 profiler 输出相对真实标注 trace 的定位、排序、归因和 actionability。

数据和互操作范围是第二个限制。本文避免同步完整测试集、图片归档和 gated 数据，因此有些来源只覆盖前缀、子配置或 redacted rows。Chrome Trace Event 和 agent-session exchange 当前只提供 exchange/reproducibility evidence，覆盖本地 session fixture 和真实标注 operation 前缀。Trace 可以作为 exchange container，但完整 OpenTelemetry/Chrome trace-ecosystem、Perfetto producer 或图像密集 benchmark 兼容仍需要单独实验。

E4 是复现和主张边界结果，不是额外的 profiler accuracy 结果。Replay、数字对齐、数据来源选择和两个抽象的表述约束让正文没有越过已有实证结果。如果采样或 oracle 本身有偏差，E4 能暴露不一致并迫使主张收窄，但不能消除偏差。因此本文主张覆盖两个抽象机制和 hidden-label pro-

filer fidelity/work tradeoff, 不覆盖 universal fixed-session dominance、automatic anomaly detection 或完整 human utility。

9 结论

本文将 agent 语义 profiling 的核心数据模型收敛为 operation 和 operation stack 两个抽象。Prompt、session、tool call、process、syscall、plan、subagent、GUI action 以及安全和质量标签, 都以 operation 的形态或字段进入同一模型。字段派生、查询过滤、权重选择、递归折叠和复现配置都只改变 operation 或查询配置。Profile spec 固定这些选择, deterministic output 让同一 profile 可以 byte-stable replay。在 15 个轻量采样的公开标注轨迹数据源上, 这个模型统一了 web、mobile、desktop、software、API、dialogue、failure、安全、human group 和 step-quality 轨迹, 并产生 flamegraph 之外的 stack tree、transition、quality、boundary、case views 和 scope reports。这个结果支持本文的核心机制主张, 即 agent profiling 的边界可以从 prompt/session/span tree 中释放出来, 成为 query-time operation-stack projection。

当前最强经验结论来自四类 oracle-rich 轨迹, 它们分别覆盖不同诊断问题。OSWorld-Human 证明同一动作序列可以折叠到人工 grouped-action 深度, AgentNet 证明 step correctness 和 redundancy 可以作为普通 operation fields 进入 profiling, AgentRewardBench 和 SATraj-OS 证明 failure、looping、side-effect、安全和 attack-type diagnostics 可以用同一机制表达。6 个 label-hidden tasks 构成主 hidden-label profiler benchmark。Query-aware operation-stack top-5 groups 用 median 0.0937 work 取得 0.188 recall, flat summary 用 1.0 work 才取得 full recall, fixed-session 用 0.0163 work 但 top-5 recall 只有 0.0233。相对 fixed-session, operation-stack top-5 recall 在 5/6 tasks 上更高, 并把 median groups 从 285.0 降到 157.5。

补充分析进一步界定这个 profiler 主张的适用条件。Task-paired uncertainty、label-permutation negative controls、view/depth task-fit、inspection-budget curves、fixed-recall target、sequence-scope scoring 和 oracle-depth scoring 共同说明, operation-stack 的主要优势是 AP/work/fragmentation tradeoff, 而不是所有指标支配。Actionability 证据进一步显示 36/36 objective rows 有具体 optimization action, 27/36 best visible policies 不是默认 operation-stack query-aware。Diagnostic lenses、case views、baseline contrasts、action counterfactuals 和 held-out transfer results 说明 profiler 暴露的是可调分析界面, 而不是 automatic universal selector。Executable profile-spec patch 进一步证明这种 profile-configuration actionability 可以落到 Rust profiler 配置上。在 6 个 patches 中, 5 个同时改善 AP、top-5 lift 和 first-positive work。OSWorld-Human reject 再由 boundary-derived operation fields 改善 AP 和 groups, 但保留 work 反例。

因此本文的新意不在单个 flamegraph, 而在把 agent 轨迹先统一成 operations, 再把边界选择推迟到 operation-stack 查询。这个接口与 pprof、Perfetto、OpenTelemetry/OpenInference 和 LLM observability systems 互补, 因为它不重新定义通用 trace schema, 也不把 dashboard 当作贡献。所有扩展都先写回 operation fields, 再由同一 operation-stack query 折叠, 包括 mapping/tagging、profile spec、agent-session trace、Chrome/Perfetto-style trace exchange 和 boundary-derived fields。这样的限制

让同一批 operations 可以按不同问题选择 view、派生字段和折叠深度, 也让读者看到每个数字对应哪一类证据。当前可写入论文的价值主张只覆盖相对 dataset-provided hidden labels 的 profiler fidelity、label-scored ranking/localization、inspection-cost/fragmentation tradeoff 和 profile-configuration actionability。它不覆盖 detector、human utility、baseline dominance、automatic patch selector 或 ecosystem compatibility。后续工作应优先加强主张所需的 oracle, 而不是无差别增加数据集。Field-derivation backend 需要跨家族 calibration, 并稳定胜过简单 sequence-derived baselines。Free-form prompt/LLM tagging 也应该先通过同一 hidden-label scoring 和 simple-baseline 检查, 再进入 profiler 主张。已有或新增真实 trace 也需要更强的 instruction-step、solution-path 或 human subtask oracle。只有当论文提出 ecosystem-baseline 主张时, 真实 OTel/Phoenix/LangSmith/Langfuse/Perfetto span-tree trace 才成为必要基线。Controlled human 或 agent analyst study 可以作为后续增强, 但不是本文主 profiling 主张的前置条件。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] OpenTelemetry GenAI semantic conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- [3] McGill-NLP WebLINX. <https://huggingface.co/datasets/McGill-NLP/WebLINX>.
- [4] OpenGVLab GUI-Odyssey. <https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey>.
- [5] WukLab OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- [6] xlangai AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.
- [7] McGill-NLP AgentRewardBench. <https://huggingface.co/datasets/McGill-NLP/agent-reward-bench>.
- [8] AI45Research SATraj-OS. <https://huggingface.co/datasets/AI45Research/SATraj-OS>.
- [9] AgentSuite tau-bench trajectories. <https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories>.
- [10] OpenGVLab ScaleCUA-Data. <https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data>.
- [11] OpenInference specification. <https://arize-ai.github.io/openinference/spec/>.
- [12] LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- [13] Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- [14] Arize Phoenix. <https://arize.com/docs/phoenix>.
- [15] AgentOps: Enabling Observability of LLM Agents. <https://arxiv.org/html/2411.05285v2>.
- [16] Brendan Gregg. The Flame Graph. <https://queue.acm.org/detail.cfm?id=2927301>.
- [17] Google pprof documentation. <https://github.com/google/pprof/blob/main/doc/README.md>.
- [18] Perfetto Trace Processor documentation. <https://perfetto.dev/docs/analysis/trace-processor>.
- [19] AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. <https://arxiv.org/abs/2504.08942>.
- [20] OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. <https://arxiv.org/abs/2404.07972>.
- [21] OpenCUA: Open Foundations for Computer-Use Agents. <https://arxiv.org/abs/2508.09123>.
- [22] Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. <https://arxiv.org/abs/2605.06230>.
- [23] AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. <https://arxiv.org/abs/2602.02475>.
- [24] Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. <https://arxiv.org/abs/2606.02060>.
- [25] Holistic Evaluation and Failure Diagnosis of AI Agents. <https://arxiv.org/abs/2605.14865>.
- [26] AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. <https://arxiv.org/abs/2605.20530>.